

AI DEBUGGER PRO

AI Debugger Pro: Human-in-the-Loop UX and Decision-Support Study Protocol



ABSTRACT

Human approval is frequently presented as the safety boundary for AI-assisted software repair, yet an approval button alone does not establish meaningful oversight. Reviewers may defer to confident explanations, miss harmful changes inside large diffs, misunderstand what “verified” means, or approve quickly under time pressure. This paper presents a preregistration-ready human-factors protocol for evaluating AI Debugger Pro, a desktop environment that executes code, captures failures, requests structured diagnoses, previews model-proposed replacement source, requires explicit approval or rejection, reruns approved candidates, and records outcomes. The study compares interface conditions that progressively add decision support: result-only output, structured diagnosis, diagnosis plus diff, and a full evidence-centered interface with scoped verification labels, provenance, risky-change warnings, and stale-proposal invalidation. Participants complete debugging tasks containing correct repairs, plausible-but-incorrect repairs, security regressions, irrelevant rewrites, low-confidence abstentions, and deceptive high-confidence explanations. Primary outcomes are decision accuracy, unsafe acceptance, beneficial rejection, calibration, review time, and comprehension of evidence scope. Secondary outcomes include cognitive workload, trust, reliance, diff inspection behavior, recovery from failed verification, and qualitative reasoning. The protocol explicitly separates measured outcomes from present repository behavior and reports no invented participant results. Its core hypothesis is that useful oversight depends less on adding friction everywhere than on placing the right evidence at the moment authority changes hands.

Keywords: human-AI interaction, automation bias, explainable AI, developer experience, code review, calibrated trust, AI-assisted repair

1. Motivation

AI Debugger Pro follows a human-governed pattern: the model proposes; the user approves or rejects; the system reruns approved code. That is a meaningful architectural safeguard. Its effectiveness nevertheless depends on human performance. If the interface makes an unsafe proposal appear authoritative, hides critical evidence, or labels a weak rerun as complete verification, the approval gate can become ceremony rather than control.

Human oversight should therefore be evaluated as a decision system. The relevant question is not “Do users like the interface?” It is “Does the interface help users accept good repairs, reject harmful ones, understand uncertainty, and recover when evidence is incomplete?” Satisfaction matters, but correctness and calibrated reliance matter more.

2. Repository-grounded interface

The Phase 6 interface presents a dark IDE-style workspace with a line-numbered editor, history panel, output area, structured diagnosis, proposed diff, regression-test tab, status indicator, and explicit apply/reject controls. `run_debug` executes current source and automatically requests diagnosis after failure. `_receive_diagnosis` stores the pending diagnosis, displays structured fields, renders a diff when replacement code exists, and enables approval. `apply_and_verify` replaces editor content, reruns it, records `verified_fix` or `failed_fix`, and clears the pending proposal. `reject_patch` records rejection without changing source.

AI work runs in a background thread, which protects interface responsiveness. Keyboard shortcuts accelerate run, diagnose, apply-and-verify, open, and save. History shows pass/fail styling and permits source restoration after confirmation.

Important present limitations shape the study. Model confidence is displayed but is self-reported. The proposal is a full-source replacement. Suggested regression tests can be saved but are not automatically executed. Positive verification is based on rerun success. Pending proposals are not cryptographically bound to immutable source and failure hashes. These are study factors, not hidden footnotes.

3. Research questions

ID	Research question
RQ1	How does progressively richer decision support affect correct repair acceptance and incorrect repair rejection?

ID	Research question
RQ2	Do scoped evidence labels reduce overinterpretation of rerun success as general correctness?
RQ3	How do model confidence and explanation fluency influence decisions when they conflict with code evidence?
RQ4	Do risky-change warnings improve security-regression detection without causing excessive rejection of safe repairs?
RQ5	How do expertise, language familiarity, and task difficulty moderate reliance on AI proposals?
RQ6	Does stale-proposal invalidation prevent approval of candidates no longer matched to the displayed source?
RQ7	Which interface elements participants inspect before accepting, rejecting, revising, or requesting more evidence?

4. Hypotheses

- **H1:** Diagnosis plus diff will outperform result-only and diagnosis-only interfaces on decision accuracy.
- **H2:** The evidence-centered condition will reduce unsafe acceptance relative to diagnosis plus diff.
- **H3:** High model confidence will increase acceptance even when proposal quality is held constant; scoped evidence will attenuate this effect.
- **H4:** Risky-change warnings will improve detection of security regressions while modestly increasing review time.
- **H5:** Stale-proposal invalidation will sharply reduce mismatched approvals without lowering correct acceptance.
- **H6:** Greater expertise will improve absolute accuracy but will not eliminate automation bias under fluent, high-confidence explanations.
- **H7:** Participants who inspect changed lines and independent test evidence will show better calibration than participants who rely primarily on summary text.

These hypotheses are directional and must be preregistered before data collection. Null, contradictory, and mixed findings remain valid outcomes.

5. Experimental design

5.1 Conditions

Condition	Information and controls
A: Result only	Failure output and candidate source; no structured explanation or confidence
B: Structured diagnosis	Summary, root cause, explanation, confidence, and candidate source
C: Diagnosis plus diff	Condition B plus unified diff and explicit approve/reject controls
D: Evidence-centered	Condition C plus scoped verification labels, provenance, risky-change annotations, test origins, and stale-proposal invalidation

A mixed design is recommended. Interface condition is between subjects to reduce learning and cross-condition contamination. Proposal type and language are within subjects, counterbalanced using a Latin-square schedule. A pilot should test timing, difficulty, and whether task defects are discoverable without ceiling or floor effects.

5.2 Participants

Recruit three experience strata: computing students or early learners; junior developers with up to two years of professional experience; and experienced developers with three or more years. Record self-rated and demonstrated familiarity for each study language rather than treating job title as perfect expertise.

Power analysis must be completed before recruitment using the primary mixed-effects model and a conservative expected effect. The protocol does not invent a sample size. Recruitment targets, exclusion criteria, compensation, and stopping rules must be preregistered. Accessibility needs and assistive-technology compatibility must not be treated as exclusions by convenience.

5.3 Assignment and masking

Randomize participants to interface conditions within experience strata. Randomize task order subject to counterbalancing. Participants cannot be masked to visible interface features, but they should not be told which proposals are adversarial or which condition is expected to perform best. Analysts should use coded conditions until primary preprocessing decisions are frozen.

6. Task corpus

The corpus should include Python, C, C++, and Java single-file tasks compatible with the current execution boundary. Each task contains source, a reproducible failure, contextual information, a candidate diagnosis, a proposed replacement, and independent ground truth established by tests and expert review.

Proposal class	Required property	Oversight decision
Correct minimal repair	Resolves defect and passes independent tests without unrelated changes	Accept
Plausible overfit	Passes observed case but fails held-out or property checks	Reject or request evidence
Exception suppression	Removes visible failure without restoring intended behavior	Reject
Security regression	Introduces network, shell, secret, filesystem, or permission risk	Reject
Irrelevant rewrite	Correct core change buried in unnecessary broad edits	Reject/revise under policy
Test weakening	Makes tests accept incorrect behavior	Reject
Honest abstention	Explains failure but supplies no patch when evidence is insufficient	Do not penalize as failed repair
Stale proposal	Candidate was generated for an earlier source version	Block or reject

Every proposal must be validated before the study. “Correct” means it satisfies the prespecified oracle and security review, not merely that it executes successfully.

7. Independent ground truth

Ground truth should combine repository-owned or expert-authored tests, hidden cases, property or metamorphic checks where appropriate, security-policy review, and independent review by at least two qualified evaluators. Disagreements are adjudicated before participant exposure. Model-generated tests may be shown as study stimuli but cannot establish the label of the proposal that generated them.

For each task, store the original failure signature, intended behavior, candidate hash, changed-line classification, test outcomes, security findings, and acceptable user decisions. Tasks with ambiguous intent should be marked as “request

clarification/evidence,” not forced into a false binary.

8. Procedure

- 1 Obtain informed consent and explain that source, interaction, and screen events may be recorded.
- 2 Administer demographic, experience, language-familiarity, and accessibility questions.
- 3 Provide standardized training on the assigned interface without exposing study tasks.
- 4 Run one practice task and comprehension check.
- 5 Present the counterbalanced task set. For each task, require accept, reject, revise/request evidence, or abstain.
- 6 Capture decision, confidence, time, inspected panels, diff navigation, evidence requests, and optional think-aloud comments.
- 7 After selected decisions, reveal verification evidence and measure whether participants appropriately revise their judgment.
- 8 Administer workload, trust, usability, and evidence-comprehension instruments.
- 9 Conduct a short interview focused on decision cues, confusing labels, ignored information, and desired evidence.
- 10 Debrief participants about deceptive proposals and study purpose.

Time limits should be generous enough to model realistic review, with time pressure evaluated only as a declared secondary condition.

9. Measures

9.1 Primary outcomes

Measure	Definition
Decision accuracy	Proportion of decisions matching prespecified safe disposition
Unsafe acceptance	Incorrect, overfit, weakened-test, stale, or security-regressing proposal accepted
Beneficial rejection	Correct minimal repair rejected without requesting useful evidence
Calibration	Relationship between decision confidence and decision correctness
Evidence-scope comprehension	Correct interpretation of what each result or verification label proves
Review time	Time from complete stimulus render to committed decision

9.2 Secondary outcomes

Collect task completion, decision changes after evidence, diff inspection depth, panel visitation, risky-change detection, test-origin recall, source-version mismatch recognition, workload, perceived usability, appropriate trust, and qualitative rationale. If eye tracking is unavailable, instrument focus, scrolling, tab selection, and dwell time as imperfect behavioral proxies.

Usability and trust scales must not replace performance outcomes. A pleasant interface that accelerates unsafe approval is a well-upholstered trapdoor.

10. Analysis plan

Use mixed-effects logistic regression for correct decisions and unsafe acceptance, with participant and task random effects. Fixed effects should include interface condition, proposal class, displayed confidence, experience stratum, language familiarity, task order, and prespecified interactions. Analyze review time with an appropriate transformed or survival model if distributions are skewed. Estimate calibration using reliability curves, Brier score, and confidence-correctness models.

Report effect sizes and uncertainty intervals, not only significance thresholds. Correct for the declared family of secondary comparisons. Missing data, technical failures, exclusions, and protocol deviations must be reported by condition. Qualitative responses should use a documented coding scheme with double coding on a prespecified subset and agreement reporting.

Primary analysis should follow intention-to-treat assignment. Per-protocol sensitivity analyses may exclude failed comprehension checks or severe technical interruptions using preregistered rules. Do not remove slow reviewers merely because caution inconveniences the mean.

11. Confounds and controls

Threat to validity	Control
Unequal task difficulty	Pilot, counterbalance, task random effects
Language familiarity	Measure directly; stratify and model interaction
Learning across tasks	Randomized order, alternate forms, order covariate
Proposal memorability	Unique surface details; no repeated defects within participant
Interface novelty	Standardized training and practice task
Demand characteristics	Neutral instructions and concealed adversarial proportions
Model-brand expectations	Hide provider identity unless it is an experimental factor
Speed-accuracy tradeoff	Report both; avoid one composite score by default
Researcher ground-truth bias	Independent tests, dual expert review, adjudication

12. Ethics, privacy, and accessibility

The study should use synthetic or consented code with no production secrets. Interaction logs may reveal skill level, reasoning, mistakes, or accessibility behavior and must be treated as participant data. Collect the minimum events needed, separate identifiers from study records, define retention and deletion, encrypt stored data, and restrict access.

Participants should know that think-aloud and screen recording are optional components where feasible. Compensation should not depend on accepting repairs or achieving high accuracy. Deceptive task framing should be limited to proposal quality, justified in review, and followed by debriefing.

The interface must support keyboard operation, readable contrast, scalable text, non-color status cues, screen-reader labeling where technically feasible, and enough time for different interaction styles. Accessibility is part of decision validity: an unreadable diff is not a neutral measurement instrument.

Institutional or ethics review requirements depend on jurisdiction and affiliation and should be resolved before recruitment.

13. Instrumentation specification

Each event should carry participant pseudonym, session, task, condition, timestamp, source/proposal hashes, interface state, and event type. Events include task render, panel open, diff navigation, evidence expansion, approval attempt, stale block, reject, revise/request evidence, verification reveal, decision change, and task completion.

Do not log raw source, traceback locals, or free text unless required by the approved protocol. Preserve event schema versions and clock behavior. Instrumentation must not alter the content, ordering, or timing of the study interface beyond negligible measured overhead.

14. Success criteria

The evidence-centered interface should be considered beneficial only if it reduces unsafe acceptance or improves overall decision accuracy without unacceptable workload or abandonment. A faster decision is not automatically better. A lower acceptance rate is not automatically safer if correct minimal repairs are broadly rejected.

Before making comparative claims, require completion of the preregistered sample, corpus validation, data-quality checks, and primary analysis. Product changes should be tied to observed failure modes: labels for comprehension failures, diff design for missed changes, provenance for misplaced confidence, and workflow gates for stale approval.

15. Repository implementation roadmap

U1 - measurement readiness

- Add immutable source, failure, proposal, and diff identifiers.
- Define stable interface-state and event schemas.
- Add privacy-preserving instrumentation behind explicit study mode.
- Create deterministic study fixtures independent of live model availability.

U2 - experimental surfaces

- Implement feature flags for the four interface conditions.
- Add scoped evidence labels and test-origin indicators.
- Add risky-change annotations and stale-proposal invalidation.
- Ensure keyboard and non-color access across conditions.

U3 - corpus and pilot

- Build validated task fixtures across all four languages.
- Complete dual expert review and hidden-test validation.
- Pilot timing, difficulty, comprehension, and logging integrity.
- Freeze preregistration, exclusions, and analysis code before recruitment.

U4 - study and release

- Run the approved study, publish anonymized materials where permitted, and report null findings.
- Translate measured interface failures into tracked engineering changes.
- Rerun critical conditions after redesign rather than treating one study as eternal truth.

16. Reporting template

A results paper should report participant flow, exclusions, task and condition counts, missing data, protocol deviations, primary effect estimates, uncertainty, calibration, workload, qualitative themes, and adverse or confusing interactions. It should include screenshots or exact interface versions, repository commit, task hashes, analysis code, and model stimulus provenance.

The title and abstract must identify the work as a protocol until data exist. After data collection, results should be published as a separate paper or clearly versioned revision; numbers should never be backfilled into this protocol without preserving the preregistered artifact.

17. Limitations

Laboratory tasks simplify real development. Participants may behave differently with their own repositories, deadlines, team norms, and accountability. Single-file defects do not represent dependency failures or large refactors. Instrumented attention does not guarantee comprehension. Experience strata cannot capture every kind of expertise. The study evaluates decision support around proposed repairs, not the total productivity of AI-assisted development.

The current paper reports no participants, outcomes, effect sizes, or conclusions about which interface wins. It defines how those claims can later be earned.

18. Conclusion

Human approval is valuable because it preserves agency, but agency without legible evidence is a thin shield. AI Debugger Pro already provides the essential interruption between proposal and mutation. The next question is whether that interruption helps people reason—or merely asks them to click.

This protocol turns oversight into a measurable engineering property. It evaluates correct acceptance, harmful rejection, calibration, comprehension, workload, and recovery across realistic deceptive proposals. Its design treats confidence as a factor, diff inspection as behavior, verification language as an intervention, and stale approval as a preventable failure. The guiding principle is simple: the interface should make careful judgment easier at the exact moment code is allowed to change.

Appendix A. Repository evidence map

UX element	Repository evidence
Editor, history, output, diagnosis, diff, test panels	interface/gui.py
Background diagnosis request	interface/gui.py::_request_diagnosis
Structured diagnosis display	interface/gui.py::_receive_diagnosis; core/diagnostics.py
Approval and rejection controls	interface/gui.py::apply_and_verify; reject_patch
Post-approval rerun and label	interface/gui.py::apply_and_verify
Regression-test display/save	interface/gui.py::save_regression_test
History restore confirmation	interface/gui.py::restore_history_entry
Keyboard shortcuts	interface/gui.py; README

Appendix B. References

1 R. Parasuraman and V. Riley. "Humans and Automation: Use, Misuse, Disuse, Abuse." *Human Factors*, 1997. <https://doi.org/10.1518/001872097778543886>

- 2 J. D. Lee and K. A. See. "Trust in Automation: Designing for Appropriate Reliance." *Human Factors*, 2004.
<https://doi.org/10.1518/0018720041855879>
- 3 M. T. Dzindolet et al. "The Role of Trust in Automation Reliance." *International Journal of Human-Computer Studies*, 2003.
[https://doi.org/10.1016/S1071-5819\(03\)00038-7](https://doi.org/10.1016/S1071-5819(03)00038-7)
- 4 A. Bussone, S. Stumpf, and D. O'Sullivan. "The Role of Explanations on Trust and Reliance in Clinical Decision Support Systems." *ICHI*, 2015. <https://doi.org/10.1109/ICHI.2015.26>
- 5 National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework*. NIST AI 100-1, 2023.
<https://doi.org/10.6028/NIST.AI.100-1>
- 6 T. R. Bentley. *AI Debugger Pro: Empirical Evaluation Protocol for Human-Governed AI-Assisted Repair*. September 2026.
- 7 T. R. Bentley. *AI Debugger Pro: Repair Verification, Evidence, and Trust*. September 2026.

Appendix C. Study artifact checklist

The study is ready to recruit only when the following artifacts are versioned, reviewed, and mutually consistent.

Artifact	Required contents	Freeze point
Preregistration	RQs, hypotheses, outcomes, exclusions, stopping rule, models	Before recruitment
Ethics packet	Consent, risks, recording, compensation, withdrawal, debrief	Before contact
Interface build	Commit, feature flags, screenshots, accessibility checks	Before pilot
Task corpus	Sources, failures, proposal classes, hashes, language balance	Before pilot
Ground truth	Tests, properties, security review, dual-rater adjudication	Before pilot
Instrumentation	Event schema, timestamps, pseudonyms, retention, failure logging	Before pilot
Randomization	Strata, condition assignment, counterbalancing seed and code	Before recruitment
Analysis package	Cleaning rules, models, contrasts, plots, sensitivity analyses	Before unblinding
Release package	Materials, anonymized data or restrictions, code, deviations	Before publication

Preflight review should confirm that all proposal labels match the independent ground truth; every condition differs only by declared interface features; no task contains real credentials or proprietary source; keyboard navigation reaches every decision control; status is not conveyed by color alone; event collection omits unnecessary raw content; technical failure produces a recoverable and reportable state; and the pilot data cannot leak into the confirmatory dataset.

The reproducibility package should identify the operating system, display scaling, application commit, Python version, model-stimulus origin, task and proposal hashes, randomization seed, analysis environment, and deviations from preregistration. When data cannot be released, publish the schema, synthetic example records, aggregation logic, and a precise access explanation. Transparency is not an all-or-nothing switch; even constrained studies can expose how the claims were manufactured.